

End-to-End Optimization

Contents

- [Content](#)
- [Motivation](#)
- [Laser protection by defocus on purpose](#)
- [Learned optimal patterns for deflectometric surface inspection](#)

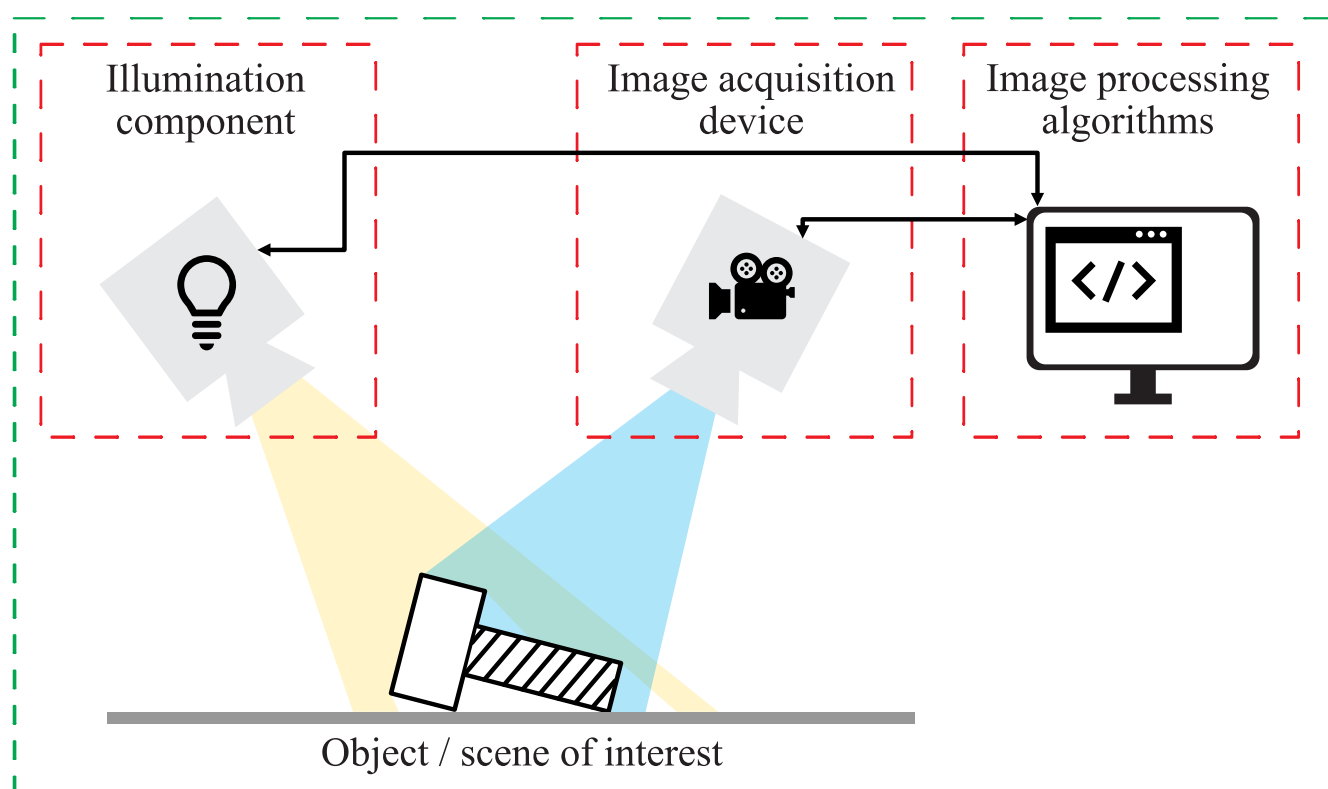
Computational Imaging

Content

- Motivation & principle idea
- Laser protection by defocus on purpose
- Learned optimal patterns for deflectometric surface inspection

Motivation

Think again about the definition of computational imaging systems we gave in the first chapter.



 conventional imaging, independent individual optimization

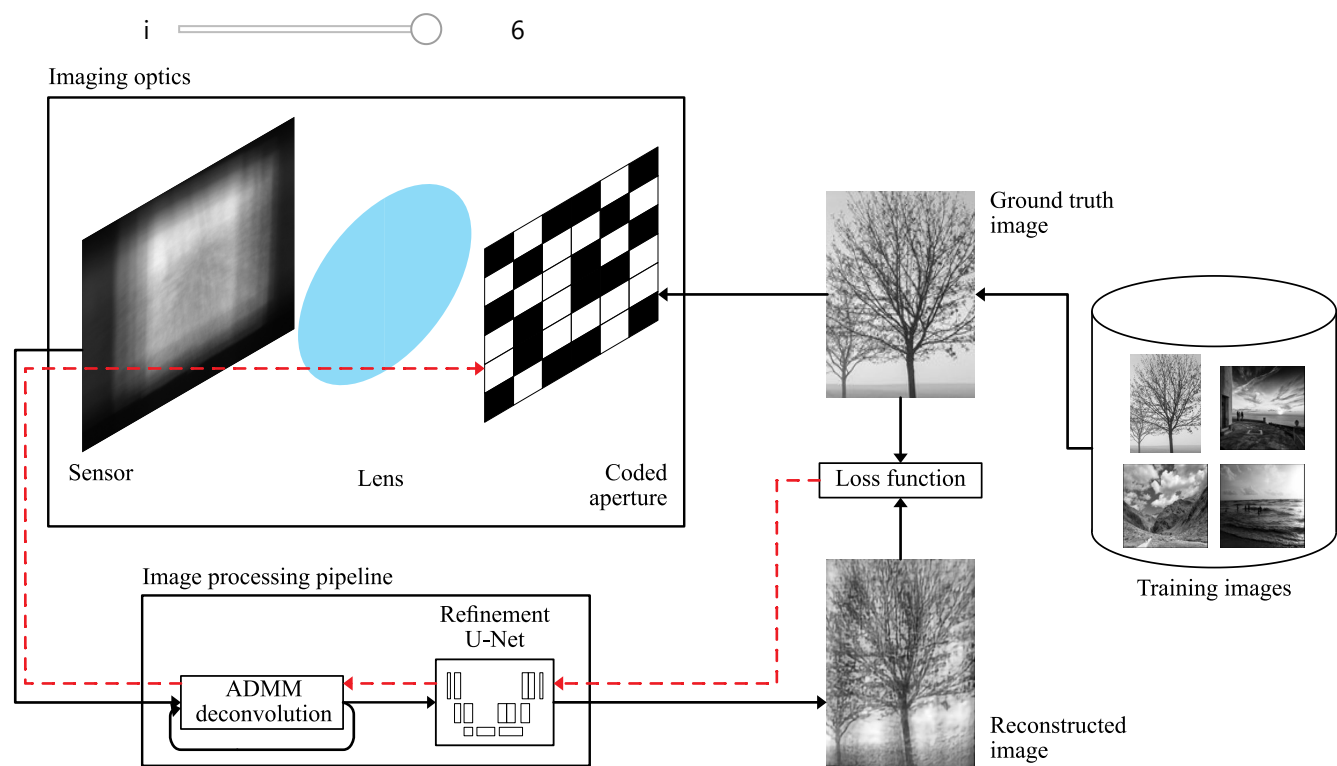
 joint optimization via computational imaging

We elaborated, that the joint optimization of all major components (illumination, image acquisition and image processing algorithms) of an artificial vision system should lead to increased performance or other advantages compared to the classical approach of separately optimizing all those components.

Principle idea

To be able to jointly optimize all those components, they have to be modeled in a common (mathematical) framework that also allows for modern optimization methods like gradient descent.

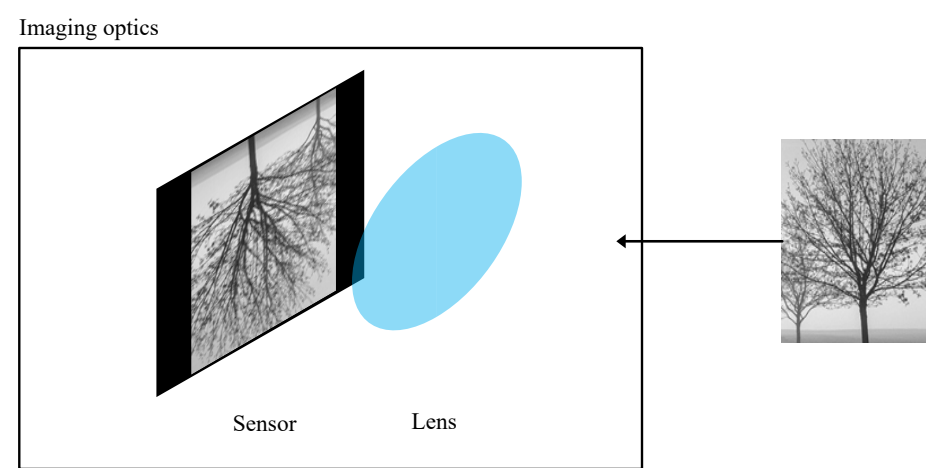
```
interact(lambda i: showFig('figures/12/end2end_general_',i,'.svg',800,50),
i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=6), step=1, value=(max_i if book else min_i)))
```



```
<function __main__.<lambda>(i)>
```

Laser protection by defocus on purpose

```
display(HTML('<img id=\"selaci\" src=\"figures/12/selaci_1.svg\" style=\"max-height:80vh\" />'))
f_dft = lambda i: Javascript(f'let image = document.getElementById(\"selaci\");
image.src = \"http://localhost:8888/files/figures/12/selaci_{i}.svg?_xsrf=\" +
globalThis.myxsrf')
interact(lambda i: f_dft(i), i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=7),
step=1, value=(max_i if book else min_i)))
```



i 7

JavaScript output is disabled in JupyterLab

```
<function __main__.<lambda>(i)>
```

The following figure shows the optical setup of the defocused imaging in detail:

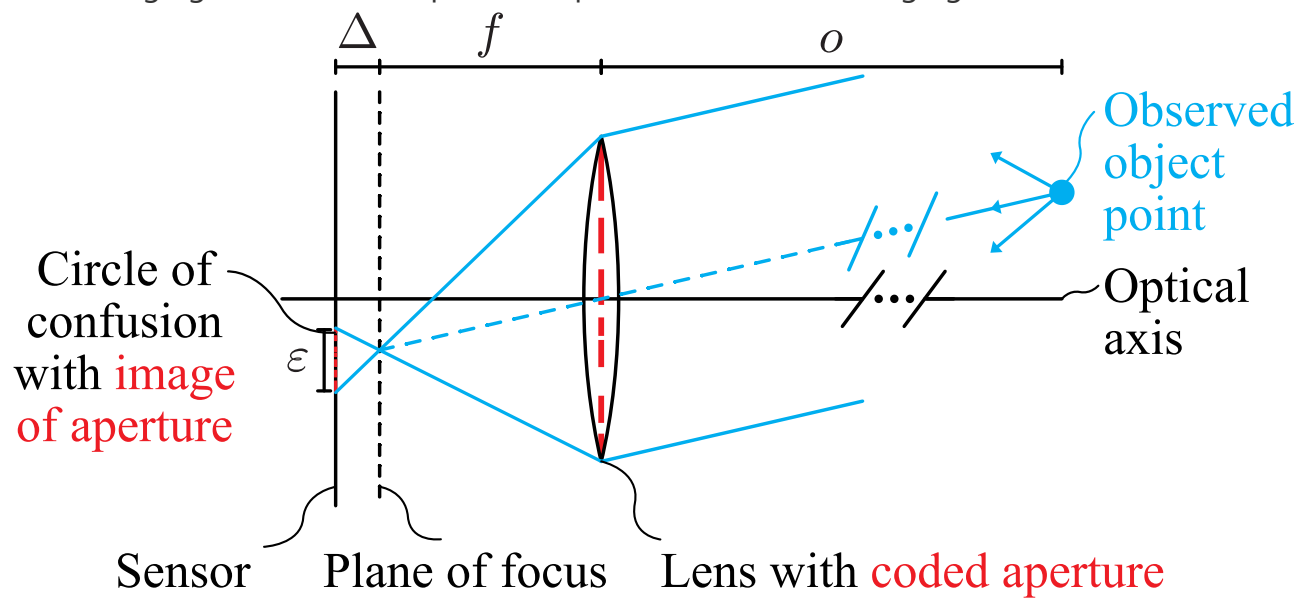


Image formation model

$$\mathbf{g} = \mathbf{H}\mathbf{s} + \mathbf{n},$$

with observation $\mathbf{g}$, matrix $\mathbf{H}$ implementing the convolution, the sought latent image $\mathbf{s}$ and the noise $\mathbf{n}$.

Image reconstruction via ADMM

Maximum a-posteriori approach

$$\hat{\mathbf{s}} = \arg \min_{\mathbf{s}} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \tau\Psi(\mathbf{s}) + \text{pos}(\mathbf{s}),$$

with balanced regularization $\tau\Psi(\mathbf{s})$ and enforced positivity $\text{pos}(\mathbf{s})$.

Introduction of slack variables

$$\begin{aligned} \hat{\mathbf{s}}, \hat{\mathbf{u}}, \hat{\mathbf{w}} &= \arg \min_{\mathbf{s}, \mathbf{u}, \mathbf{w}} \frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 + \tau\Psi(\mathbf{u}) + \text{pos}(\mathbf{w}) \\ \text{s. t. } \quad &\mathbf{s} - \mathbf{u} = \mathbf{0}, \quad \mathbf{s} - \mathbf{w} = \mathbf{0}, \end{aligned}$$

with slack variables $\mathbf{u}, \mathbf{w}$.

Transformation into augmented Lagrangian in scaled form

$$\begin{aligned} L_{\tau, \alpha, \beta}(\mathbf{s}, \mathbf{u}, \mathbf{w}, \tilde{\mathbf{u}}, \tilde{\mathbf{w}}) &= \frac{1}{2} \|\mathbf{H}\mathbf{s} - \mathbf{g}\|_2^2 \\ &+ \tau\Psi(\mathbf{u}) + \frac{\alpha}{2} \|\mathbf{s} - \mathbf{u} + \tilde{\mathbf{u}}\|_2^2 + \frac{\alpha}{2} \|\tilde{\mathbf{u}}\|_2^2 \quad \text{with the dual variables } \tilde{\mathbf{u}}, \tilde{\mathbf{w}} \text{ of } \mathbf{u}, \mathbf{w} \text{ and the} \\ &+ \text{pos}(\mathbf{w}) + \frac{\beta}{2} \|\mathbf{s} - \mathbf{w} + \tilde{\mathbf{w}}\|_2^2 + \frac{\beta}{2} \|\tilde{\mathbf{w}}\|_2^2, \end{aligned}$$

corresponding weights α, β .

Anisotropic total variation regularization

$$\Psi(\mathbf{s}) = \|\mathbf{D}\mathbf{s}\|_1, \text{ with finite difference operator } \mathbf{D} = [\mathbf{D}_x^T \mathbf{D}_y^T]^T \in \mathbb{R}^{2N \times N}.$$

Update rules

$$\begin{aligned} \mathbf{s}_{i+1} &\leftarrow (\mathbf{H}^T \mathbf{H} + \alpha \mathbf{D}^T \mathbf{D} + \beta \mathbf{I})^{-1} \\ &\quad \cdot (\mathbf{H}^T \mathbf{g} - \alpha \mathbf{D}^T (\tilde{\mathbf{u}}_i - \mathbf{u}_i) - \beta (\tilde{\mathbf{w}}_i - \mathbf{w})), \\ \mathbf{u}_{i+1} &\leftarrow \mathcal{S}_{\tau/\alpha}(\mathbf{D}\mathbf{s}_{i+1} + \tilde{\mathbf{u}}_i), \\ \mathbf{w}_{i+1} &\leftarrow \max(\mathbf{0}, \mathbf{s}_{i+1} + \tilde{\mathbf{w}}_i), \\ \tilde{\mathbf{u}}_{i+1} &\leftarrow \tilde{\mathbf{u}}_i + \mathbf{D}\mathbf{s}_{i+1} - \mathbf{u}_{i+1}, \\ \tilde{\mathbf{w}}_{i+1} &\leftarrow \tilde{\mathbf{w}}_i + \mathbf{s}_{i+1} - \mathbf{w}_{i+1}, \end{aligned}$$

denoting the identity matrix and the element-wise soft thresholding operator $\mathcal{S}_{\tau/\alpha}$.

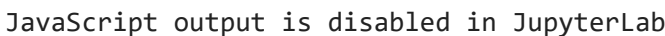
Problem: cyclic convolution assumption

Question

Optical zero padding


$$d_{\text{f}} = 2 \left(\frac{\frac{d_{\text{L}} + d_{\text{s}}}{2}}{f + \Delta} \cdot f - \frac{d_{\text{L}}}{2} \right).$$

```
display(HTML('<img id=\\"selaci_optPipe\\" src=\\"figures/12/selaci_optPipeline_1.svg\\" style=\\"max-height:80vh\\" />'))
f_optPipe = lambda i: Javascript(f'let image =
document.getElementById(\\"selaci_optPipe\\"); image.src =
\\"http://localhost:8888/files/figures/12/selaci_optPipeline_{i}.svg?_xsrf=\\" +
globalThis.myxsrf')
interact(lambda i: f_optPipe(i), i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=6),
step=1, value=(max i if book else min i)))
```



```
<function __main__.<lambda>(i)>
```

Optimization of the PSF

Constraint for light efficiency:

? Question

Do you have an idea how a loss function for the PSF could look like that is minimized when a certain user-definable fraction p of light is transmitted by the amplitude mask?

$$\ell_{\text{PSF}}(h) = \left(\frac{\sum_{\mathbf{x} \in \Omega} h(\mathbf{x})}{|\Omega|} - p \right)^2,$$

with the discrete spatial support Ω of the PSF and the sought light efficiency ratio $p \in (0, \dots, 1)$.

This loss term reaches its minimum when the mean of the coefficients of the PSF approaches p , i.e., when a ratio p of the incident light is transmitted.

Binary coefficients

? Question

What could be done to force the values of the PSF to be close to either 0 or 1? Think about a function you got to know that processes its input in that manner ...

Optimize intermediate variable $\tilde{h}$ instead of h directly.

Feed $\tilde{h}$ into steep sigmoid function:

$$h(\mathbf{x}) \leftarrow \frac{1}{1 + \exp(-\gamma \tilde{h}(\mathbf{x}))},$$

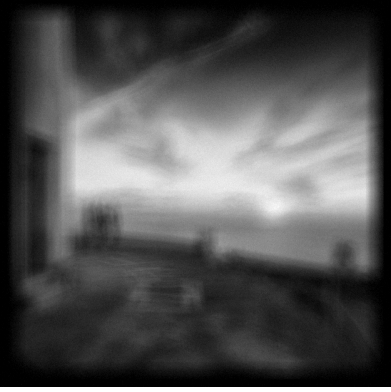
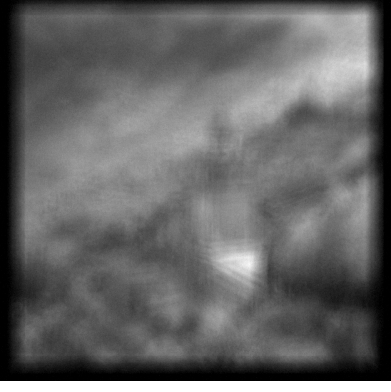
with a scaling factor γ of, e.g., $\gamma = 20$.
Then h will be approximately binarized.

Results on simulated data

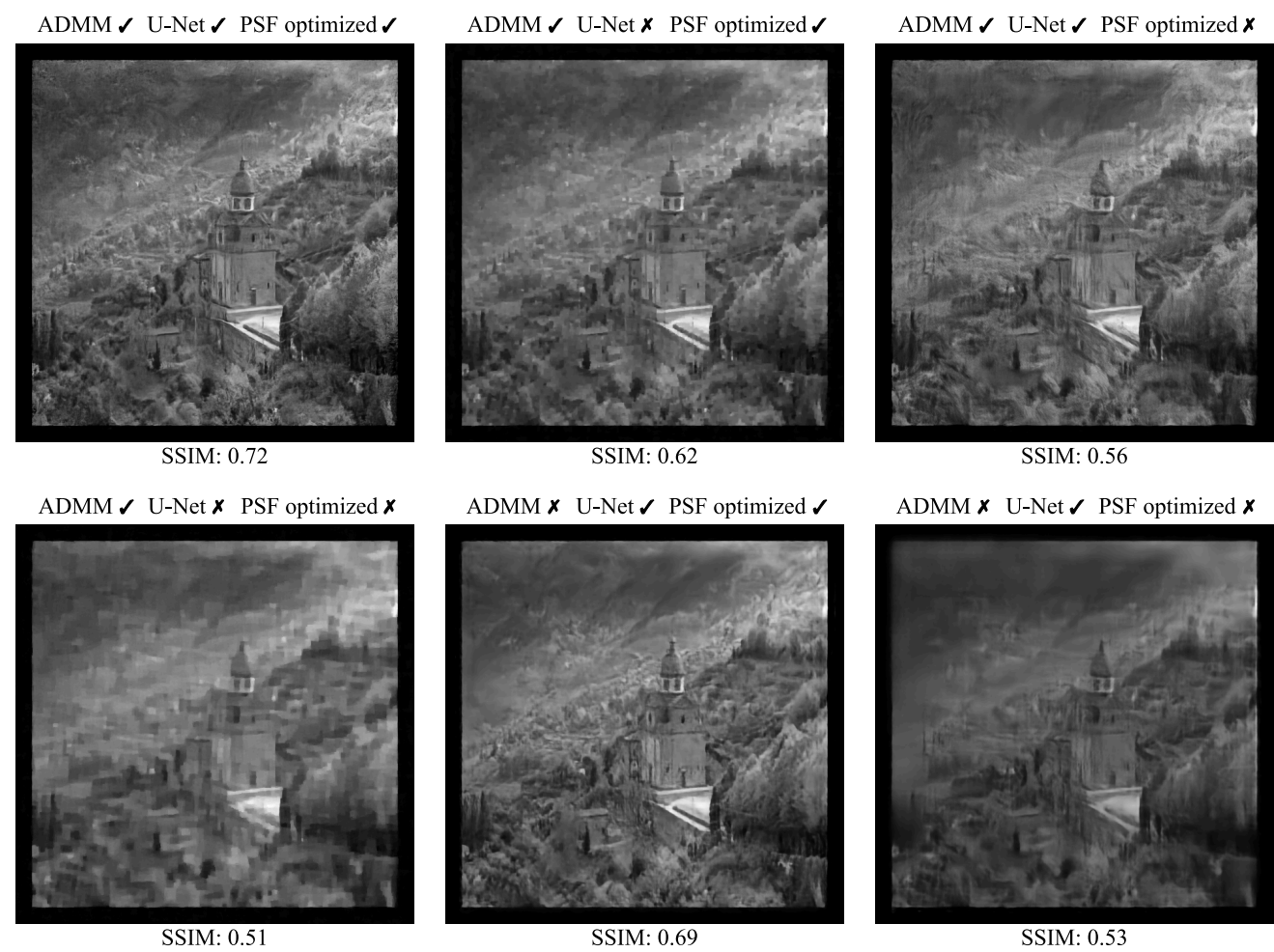
Number of ADMM iterations



Simulated reconstruction results

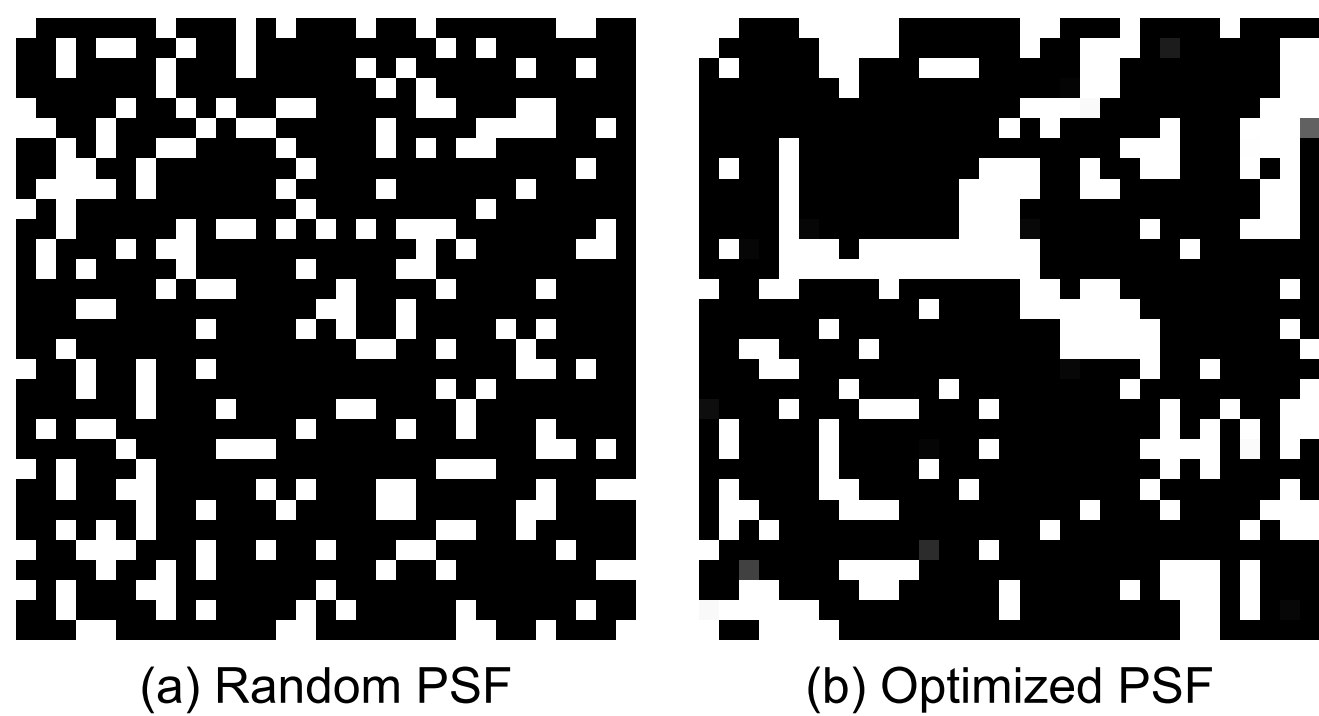
	Noisy observations	Ground truth images	Reconstructions	SSIM
Image 1				0.74
Image 2				0.75
Image 3				0.74
Image 4				0.72

Ablation studies



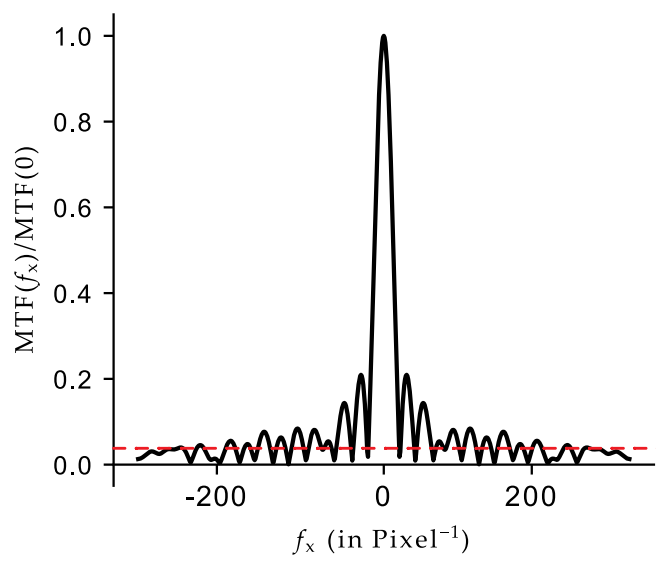
ADMM	U-Net	PSF optimized	MSE $\pm \sigma$
✓	✓	✓	131.52 $\pm$ 3.67
✓	✗	✓	145.31 $\pm$ 0.71
✓	✓	✗	193.25 $\pm$ 4.10
✓	✗	✗	211.32 $\pm$ 0.21
✗	✓	✓	158.93 $\pm$ 7.23
✗	✓	✗	218.90 $\pm$ 3.01

Influence of the PSF optimization



? Question

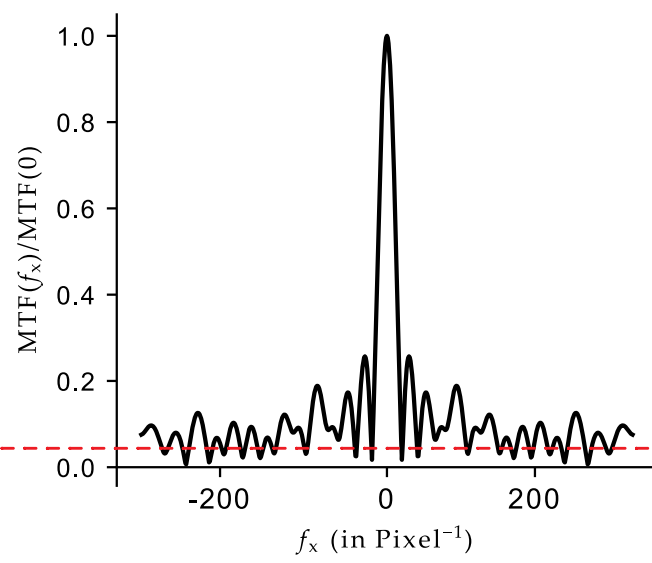
How could we get a more system-theoretical insight about what those PSFs are capable of?



(a) MTF of unoptimized, random PSF

MTF of unoptimized, random PSF

image-5.png



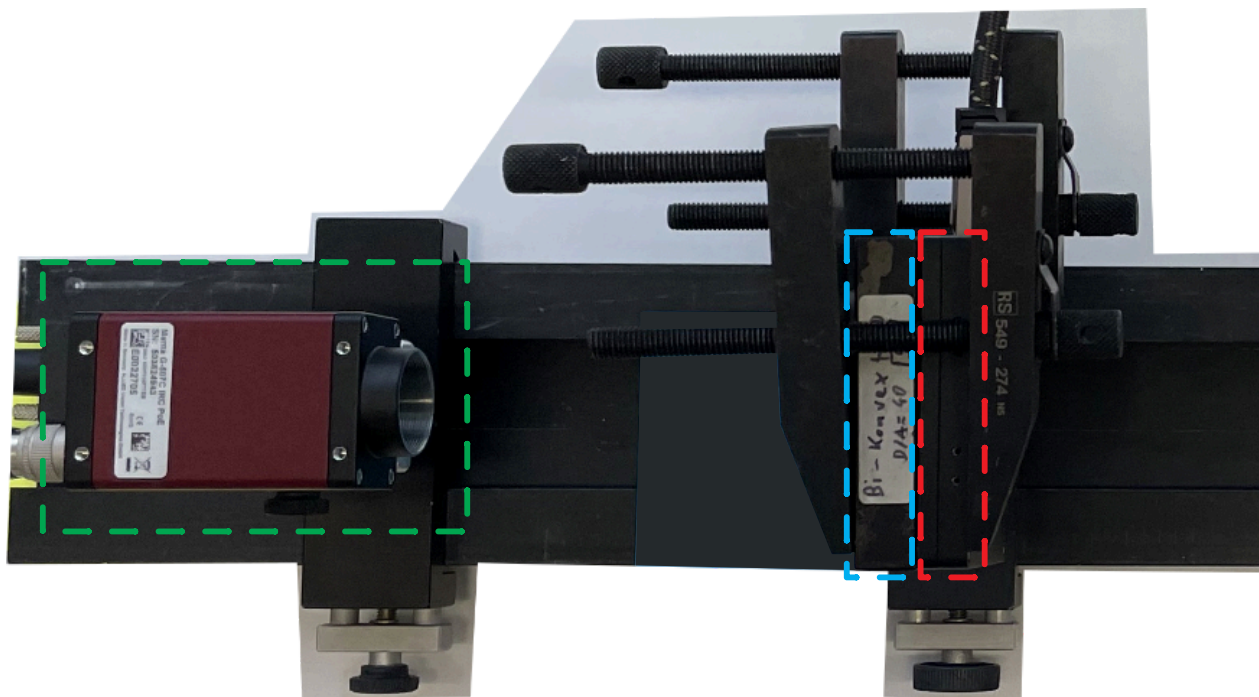
(a) MTF of optimized PSF

MTF of optimized PSF

image-6.png

Results on real data

Amplitude SLM

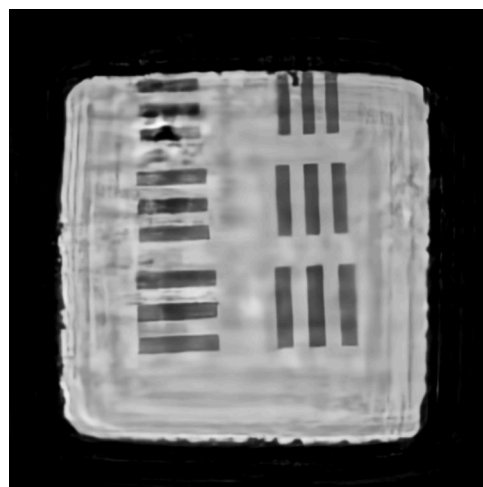


Camera

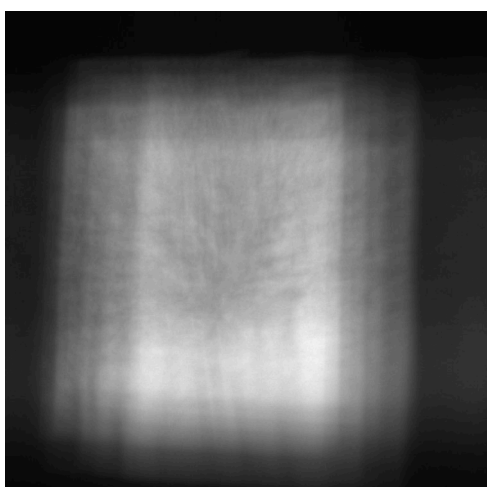
Double convex lens



(a) Observation of bar test target



(b) Reconstruction based on (a)



(c) Observation of printed tree image



(d) Reconstruction based on (c)

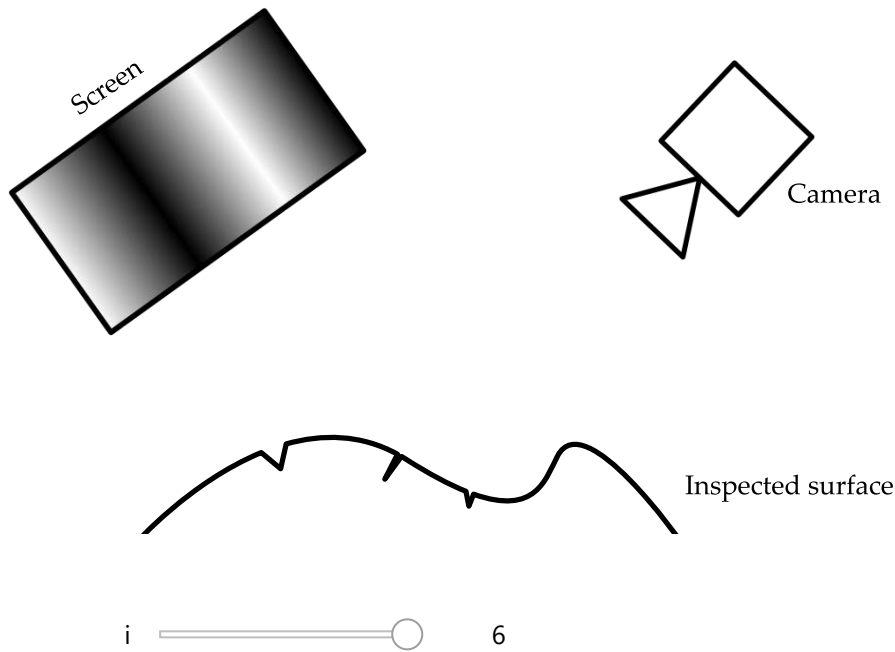
Achieved robustness

About 1,600 times more robust, i.e., a potential increase of laser power of 350 W.

Learned optimal patterns for deflectometric surface inspection

Classical deflectometry

```
display(HTML('<img id=\"deflecto\" src=\"figures/12/deflecto_1.svg\" style=\"max-height:80vh\" />'))
f_deflecto = lambda i: Javascript(f'let image = document.getElementById(\"deflecto\"); image.src = \"http://localhost:8888/files/figures/12/deflecto_{i}.svg?_xsrf=\" + globalThis.myxsrf')
interact(lambda i: f_deflecto(i), i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=6), step=1, value=(max_i if book else min_i)))
```



JavaScript output is disabled in JupyterLab

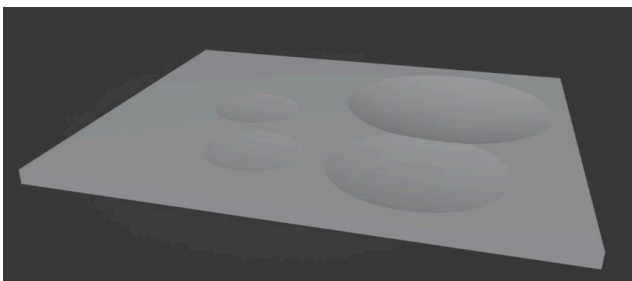
```
<function __main__.<lambda>(i)>
```

- ⚡ Screen patterns are not optimized but chosen heuristically!
- 💡 Idea: Optimize pattern in end-2-end manner via learning.

Create synthetic test object with random defects

```
display(HTML('<img id=\"deflecto_defects\" src=\"figures/12/deflecto_defects_1.svg\" style=\"max-height:80vh\" />'))
f_deflecto_defects = lambda i: Javascript(f'let image = document.getElementById(\"deflecto_defects\"); image.src = \"http://localhost:8888/files/figures/12/deflecto_defects_{i}.svg?_xsrf=\" + globalThis.myxsrf')
interact(lambda i: f_deflecto_defects(i), i=widgets.IntSlider(min=(min_i:=1),max=(max_i:=3), step=1, value=(max_i if book else min_i)))
```

Test object: mirror with parabolic valleys

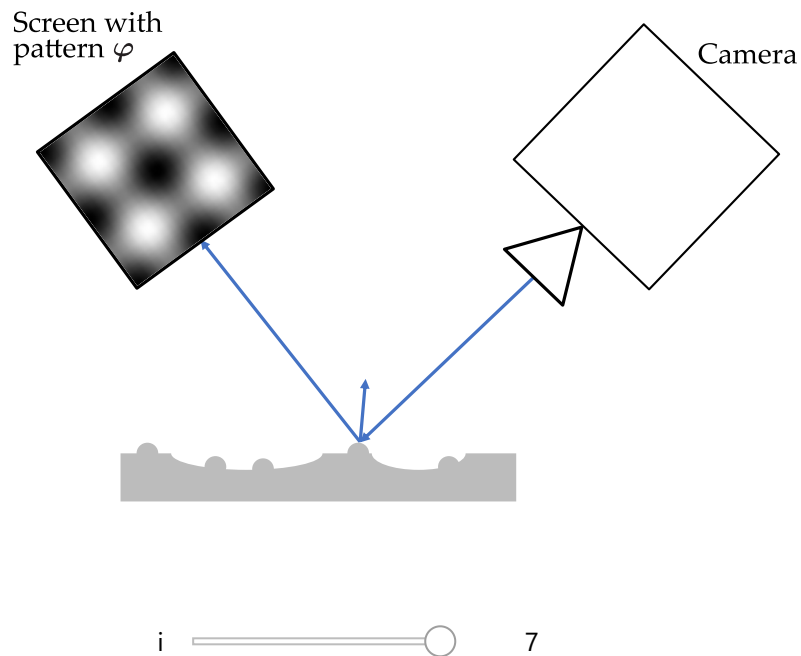


JavaScript output is disabled in JupyterLab

```
<function __main__.<lambda>(i)>
```

Optimize pattern via differentiable inverse rendering

```
display(HTML('<img id=\"deflecto_opt_pipeline\"  
src=\"figures/12/deflecto_opt_pipeline_1.svg\" style=\"max-height:80vh\" />'))  
f_deflecto_opt_pipeline = lambda i: Javascript(f'let image =  
document.getElementById(\"deflecto_opt_pipeline\"); image.src =  
\"http://localhost:8888/files/figures/12/deflecto_opt_pipeline_{i}.svg?_xsrfrf=\" +  
globalThis.myxsrfrf')  
interact(lambda i: f_deflecto_opt_pipeline(i), i=widgets.IntSlider(min=  
(min_i:=1),max=(max_i:=7), step=1, value=(max_i if book else min_i)))
```



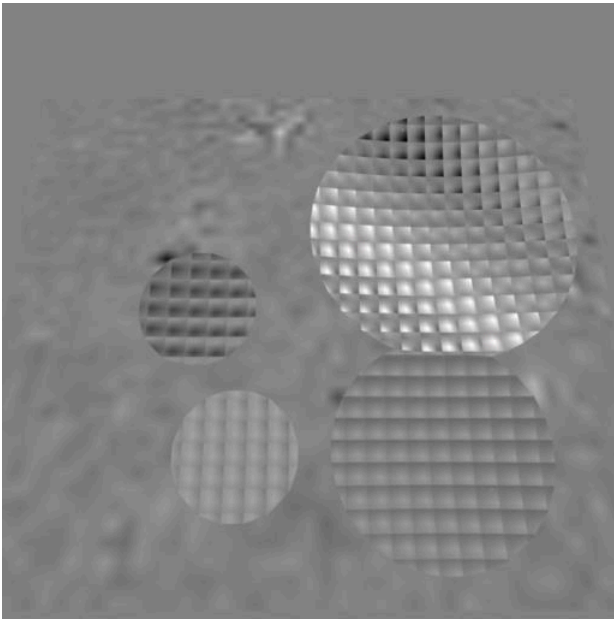
JavaScript output is disabled in JupyterLab

```
<function __main__.<lambda>(i)>
```

Optimization result

```
if book:  
    display(HTML('<img id=\"deflecto_pat_res\"  
src=\"figures/12/deflecto_pat_res_2.svg\" style=\"max-height:80vh\" />'))  
else:  
    display(HTML('<img id=\"deflecto_pat_res\"  
src=\"figures/12/deflecto_pat_res_1.svg\" style=\"max-height:80vh\" />'))  
    f_deflecto_pat_res = lambda i: Javascript(f'let image =  
document.getElementById(\"deflecto_pat_res\"); image.src =  
\"http://localhost:8888/files/figures/12/deflecto_pat_res_{i}.svg?_xsrfrf=\" +  
globalThis.myxsrfrf')  
    interact(lambda i: f_deflecto_pat_res(i), i=widgets.IntSlider(min=(min_i:=1),max=  
(max_i:=2), step=1, value=(max_i if book else min_i)))
```

Camera image



Optimized pattern

